EECS 111 System Software, Spring 2026
Project 2: Thread & Synchronization
Due Date (May 16, 2026 11:59 PM)

Instructions

In this project, you will design and implement a thread synchronization solution for a shared Specialized Resource Room. The starter code provides you with a small `pthread` example, a `Person` class, a `ResourceRoom` class skeleton, utility functions, and a Makefile. Your task is to implement the input-generation loop, queue management, and synchronization policy.

Starter Code Files

After unzipping the starter-code directory, you should see the following files:
1. main.cpp: top-level program entry point
2. p2_threads.h: thread function declarations
3. p2_threads.cpp: thread function implementation file
4. utils.h: utility function declarations
5. utils.cpp: utility function implementation file
6. types_p2.h: Person and ResourceRoom class declarations
7. types_p2.cpp: Person and ResourceRoom class implementation file
8. Makefile: compilation script

You need to complete main.cpp, p2_threads.cpp, and types_p2.cpp. You may also modify the header files and add new source or header files if needed. Your code must compile by running `make`, and the executable must be named `p2_exec`. Use `make clean` to remove compiled files before preparing your submission.

Command-Line Argument

Your executable takes exactly one positive integer argument, N. If the argument is missing or invalid, print the following format:

[ERROR] Expected 1 argument, but got (X)
[USAGE] p2_exec <number>

Problem Description

The university has opened a new Specialized Resource Room containing sensitive equipment. Because the room is small and the workflows of different groups are very different, the university has established access rules for Faculty Members and Students.

- Group Harmony: If a Faculty Member is currently in the room, other Faculty Members may enter and work alongside them. Students are not allowed to enter at that time.
- Student Access: If a Student is currently in the room, other Students may enter and work alongside them. Faculty Members are not allowed to enter at that time.

A digital sign outside the door indicates exactly one of these room states:

- Empty
- FacultyInside
- StudentInside

Required Functions

Your program must define and use the following four `ResourceRoom` functions:

```
faculty_wants_to_enter(Person& p)
student_wants_to_enter(Person& p)
faculty_leaves(Person& p)
student_leaves(Person& p)
```

You may add helper functions, counters, queues, class members, and synchronization objects, but these four functions must exist and must be used by the per-person threads. Each function should help maintain the Resource Room state and enforce the access rules.

Threading Requirements

Your program must create one pthread for each generated person. Each person thread should:
1. attempt to enter the Resource Room using the appropriate wants_to_enter function,
2. block if the current room state does not allow that person's group to enter,
3. stay in the Resource Room for that person's assigned stay time,
4. leave using the appropriate leaves function.

The main thread should generate people, create their threads, and join all person threads before exiting. The main thread should not do the work of entering, waiting inside, or leaving on behalf of a person.

Waiting threads must block using synchronization primitives such as condition variables or semaphores. Repeated polling loops with sleep are considered busy waiting and will not receive full credit.

Workload Generation

For input N, your program must generate exactly 2N total people: N Faculty Members and N Students. The order of the generated people must be randomized while preserving exactly N people from each group.

After each new person is generated, wait for a random interval from 1 ms to 5 ms before generating the next person. Each person must also receive a random Resource Room stay time from 3 ms to 10 ms. Your program should generate a different sequence and different stay times across different runs.

Runtime Output

Your program must print timestamps in milliseconds relative to the start of the program. Keep the required prefixes and role/state strings exactly as shown below:

- Faculty Member
- Student
- Resource Room
- (Empty)
- (FacultyInside)

- (StudentInside)

Your program must report input events, queue status, room entry, room exit, and final summary information. The queue-status line is required after each input event.

Required output formats:
[<Time> ms][Input] A person (<Faculty Member|Student>) goes into the queue.
[<Time> ms][Queue] Waiting queue is <empty|not empty>. Status: Total: <number> (Faculty: <number>, Students: <number>)
[<Time> ms][Queue] Send (<Faculty Member|Student>) into the resource room (Stay <number> ms), Status: Total: <number> (Faculty: <number>, Students: <number>)
[<Time> ms][Resource Room] (<Faculty Member|Student>) goes into the resource room, State is (<FacultyInside|StudentInside>): Total: <number> (Faculty: <number>, Students: <number>)
[<Time> ms][Resource Room] (<Faculty Member|Student>) left the resource room. <State is|Status is changed, Status is> (<Empty|FacultyInside|StudentInside>) : Total: <number> (Faculty: <number>, Students: <number>)
[<Time> ms][Summary] Finished all <number> people. Total runtime: <number> ms

Example output fragment:
[01 ms][Input] A person (Faculty Member) goes into the queue.
[01 ms][Queue] Waiting queue is not empty. Status: Total: 1 (Faculty: 1, Students: 0)
[12 ms][Queue] Send (Faculty Member) into the resource room (Stay 4 ms), Status: Total: 0 (Faculty: 0, Students: 0)
[12 ms][Resource Room] (Faculty Member) goes into the resource room, State is (FacultyInside): Total: 1 (Faculty: 1, Students: 0)
[16 ms][Resource Room] (Faculty Member) left the resource room. Status is changed, Status is (Empty) : Total: 0 (Faculty: 0, Students: 0)
[45 ms][Summary] Finished all 20 people. Total runtime: 45 ms

The exact timestamps and ordering will vary because your program is randomized and concurrent. The format and required strings must remain consistent.

Correctness Expectations

Your solution must satisfy all of the following:
1. no Faculty Member and Student may be inside the Resource Room at the same time,
2. multiple people from the same group may be inside together,
3. waiting people must eventually get a fair chance to enter,
4. the program must terminate after all generated people leave,
5. the output must make the queue and room states understandable to a grader.

Your code must show parallel behavior. In some runs, graders should be able to observe the room total become greater than 1 for the same group, such as Total: 2 (Faculty: 2, Students: 0) or Total: 2 (Faculty: 0, Students: 2). Small inputs may not always show sharing due to random timing, so test with larger inputs such as ./p2_exec 10.

Conceptual Questions

Submit brief answers to the following questions with your project in `questions.txt` file:

1. Describe how your design prevent Faculty Members and Students from being inside the room at the same time. Also, describe how you are protecting read and write of shared variables.
2. Are you using busy waiting in your program? Please point out in your code on how the program is using busy waiting or avoiding it.
3. How did you prevent deadlock, which makes program run forever? Also, how did you reduce the chance of starvation so that program runs quickly?

Submission

You can compress all the files into a single archive .zip file or upload all files together to Gradescope. The deadline for submission is May 16, 2026 11:59 PM. Since submissions may be processed by a program, follow these rules carefully:

1. Keep the required output format exactly as specified.
2. Use only the C++98 standard. Do not use C++11, C++14, or newer language features.
3. Make sure your code compiles and runs with make.
4. Your executable filename must be p2_exec.
5. Your code must show parallel behavior.
6. Your submission must contain all source code files and the Makefile in the root of the uploaded zip file or Gradescope upload.
7. If creating a zip file, run the command from inside the directory containing your source code:
zip -j p2.zip <list of your source code files>
Example:
zip -j p2.zip main.cpp p2_threads.h p2_threads.cpp types_p2.h types_p2.cpp utils.h utils.cpp Makefile
8. Your code will be checked for similarity against other submissions. You may discuss ideas and use course resources, but you must write your own code.

Grading

1. (5%) Following the submission format
2. (10%) Compiling with the provided Makefile without errors
3. (10%) Command-line output matches the required format for different inputs
4. (5%) Randomly generating the input sequence and Resource Room stay times
5. (15%) Running with one pthread per person and terminating without deadlock
6. (10%) Disallowing different-group people and allowing multiple same-group people to be in the Resource Room at the same time
7. (10%) Performing synchronization with no busy waiting
8. (5%) Avoiding starvation
9. (30%) Answers to conceptual questions

Note

- Even if your output looks correct once, your code may still fail on timing corner cases.
- Run your program multiple times and test different input sizes.
- If you have questions, ask them in the discussion session.